Name :- Aaryan N. Rao Rane Roll No :- 5004 Div :- S batch :- A1
branch :- AI&DS SAP ID :- 60018250070

Experiment No 10

22/10

Title :- File Handling and Exception Handling.

Aim :- To understand and apply file handling and Exception handling in Python.

Theory :-

Q1 What is an Exception? How is it different from an error?

⟹ An Exception is an event that occurs while a program is running and disturbs its normal flow, for Example dividing a number by zero or opening a file that does not exist. When an Exception occurs, Python stops the normal execution and looks for a block of code that can handle it. If no such block is found, the program terminates with a message. An Error is a broader term and a common kind of error is a syntax mistake, which is found before the program starts and cannot be handled during Execution.

Point	Error	Exception.
Meaning	A mistake in the program that stops it from running	An unexpected event that occurs during execution.
When it occurs	Before execution for syntax errors	While the program is running.

Sundaram
FOR EDUCATIONAL USE

Handling	cannot be handled, the code must be corrected	can be handled using try and except.
Example	Missing colon after an if statement	ZeroDivisionError, FileNotFoundError.

code:-
```
try:
    a = 10
    b = 0
    print(a/b)
except ZeroDivisionError:
    print("Exception handled: division by zero is not possible")
print("Program continues after handling")
```

output:-

Exception handled: division by zero is not possible
Program continues after handling.

Q2 List and describe different file opening modes used in Python

=> The mode is given as the second argument of open() and decides whether the file is read, written or appended, and what happens if the file does not exist.

Sundaram FOR EDUCATIONAL USE

Mode	Description	If file does not exist
r	opens the file for reading only, pointer at the start.	Raises FileNotFoundError
w	Opens the file for writing, old contents erased	creates a new file
a	Opens the file for appending, new data goes at the end.	creates a new file.
r+	Opens the file for both reading and writing.	Raises FileNotFoundError
w+	Opens for writing and reading, old contents erased.	creates a new file.
a+	Opens for appending and reading.	creates a new file.
rb, wb	Binary versions of r and w, used for images and similar files	Same as r and w

Sundaram®
FOR EDUCATIONAL USE

code:

```
f = open("demo.txt", "w")
f.write("first line\n")
f.close()

f = open("demo.txt", "a")
f.write("second line\n")
f.close()

f = open("demo.txt", "r")
print(f.read())
f.close()
```

output:

```
First line
second line.
```

Q3 what is the use of the close() function in file handling?

⇒ The close() function closes an open file. closing makes sure that any data still waiting in the Buffer is written to the disk and frees the resources that the operating system had given to the file. A file that is left open may lose data, and it can also stay locked so that other programs cannot use it.

Sundaram®
FOR EDUCATIONAL USE

code :-

```
f = open("notes.txt", "w")
f.write("Data saved to disk")
print("closed before close(): " + str(f.closed))
f.close()
print("closed after close(): " + str(f.closed))
```

output :-

```
closed before close(): False
closed after close(): True
```

Q4 Differentiate between read(), readline(), and readlines() methods.

=>

Method	what it returns	Type of result.
read()	The entire content of the file, or the given number of characters	One string
readline()	Only the next single line from the file	One string.
readlines()	All the lines of the file, one element per line	A list of strings.

Sundaram®
FOR EDUCATIONAL USE

The function repr() is used below only to make the newline characters visible in the output.

Code :-

```
f = open("sample.txt", "w")
f.write("Apple\n Banana\n Cherry\n")
f.close()

f = open("sample.txt", "r")
print(repr(f.read()))
f.close()

f = open("sample.txt", "r")
print(repr(f.readline()))
f.close()

f = open("sample.txt", "r")
print(f.readlines())
f.close()
```

Output :-

```
'Apple\n Banana\n Cherry\n'
'Apple\n'
['Apple\n', 'Banana\n', 'Cherry\n']
```

Sundaram
FOR EDUCATIONAL USE

Q5 What is the function of write() and writelines() methods? Give examples.

=> The write() method writes a single string to the file and returns the number of characters written. The writelines() method writes every string of a list or other sequence to the file in one call. Neither method adds a newline by itself, so the newline character must be included in the text

Code:

```
f = open("out.txt", "w")
n = f.write("Hello World\n")
print("Characters written:" + str(n))

lines = ["Python\n", "File handling\n", "Exceptions\n"]
f.writelines(lines)
f.close()

f = open("out.txt", "r")
print(f.read())
f.close
```

Output:

```
Characters written: 12
Hello World
Python
File handling
Exceptions
```

Sundaram
FOR EDUCATIONAL USE

Q6 What is the purpose of the seek() and tell() methods?

⇒ Every open file has a pointer that marks the position from where the next read or write will happen. The tell() method returns the current position of this pointer as a number of bytes from the start of the file. The seek() method moves the pointer to a chosen position, and seek(0) takes it back to the beginning so that the file can be read again without reopening it.

code:

```
f = open("sample.txt", "r")
print(f.tell())
print(f.read(5))
print(f.tell())
f.seek(0)
print(f.tell())
print(f.readline())
f.close()
```

output:

```
0
apple
5
0
apple
```

Sundaram®
FOR EDUCATIONAL USE

Q7 What is the use of the with statement in file handling? How does it simply simplify file operations?

=> The with statement opens a file and closes it automatically when the block ends, even if an exception occurs inside the block. This removes the need to call close() by hand and avoids the risk of forgetting it. The code also becomes shorter and easier to read, because the work on the file stays together inside one indented block.

code:-

```
with open("sample.txt", "r") as f:
    for line in f:
        print(line.strip())
print("File closed automatically:" + str(f.closed))
```

output:-

```
Apple
Banana
Cherry
File closed automatically: True
```

Sundaram
FOR EDUCATIONAL USE

Conclusions:

File handling in Python lets a program store data permanently by opening, reading, writing and closing files using the correct mode. Exception handling with try, except and finally allows the program to deal with problems such as a missing file without stopping abruptly. The with statement combines both ideas by closing the file safely, which makes the code cleaner and more reliable.

Sundaram®
FOR EDUCATIONAL USE